GPU – MEMORY TYPES AND STREAM

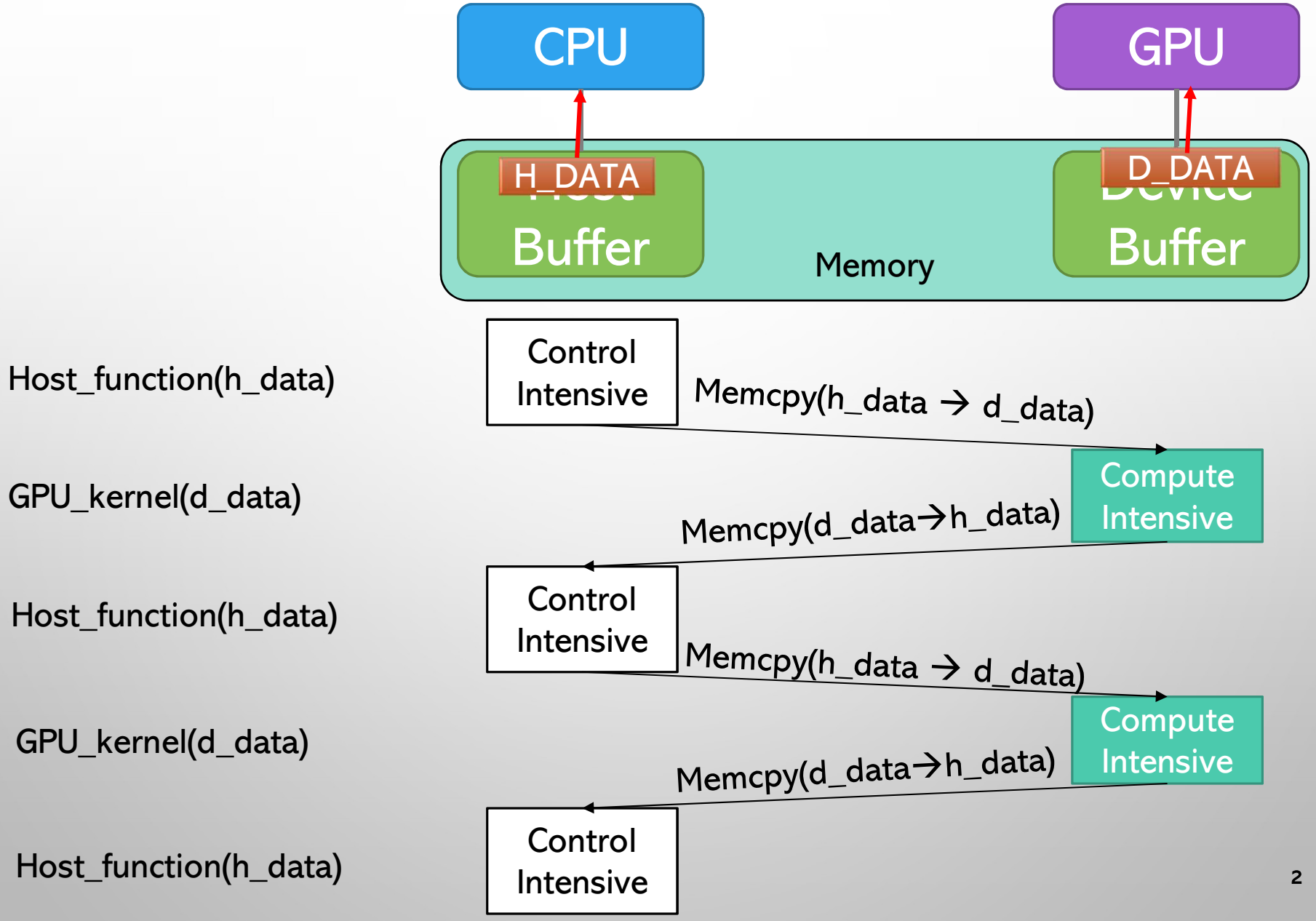
2026 SPRING

JINHO LEE

ECE, SNU

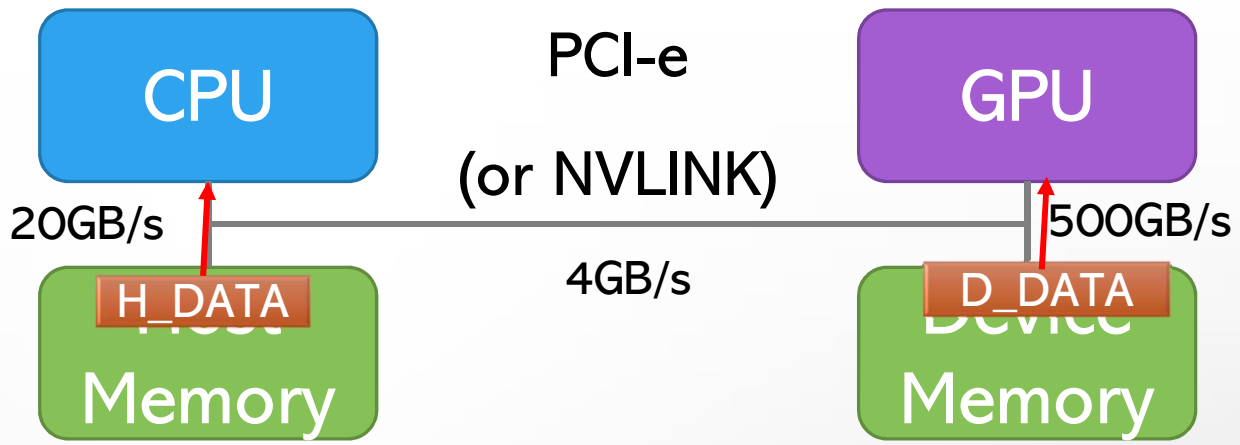
Accelerated Intelligent Systems Lab.

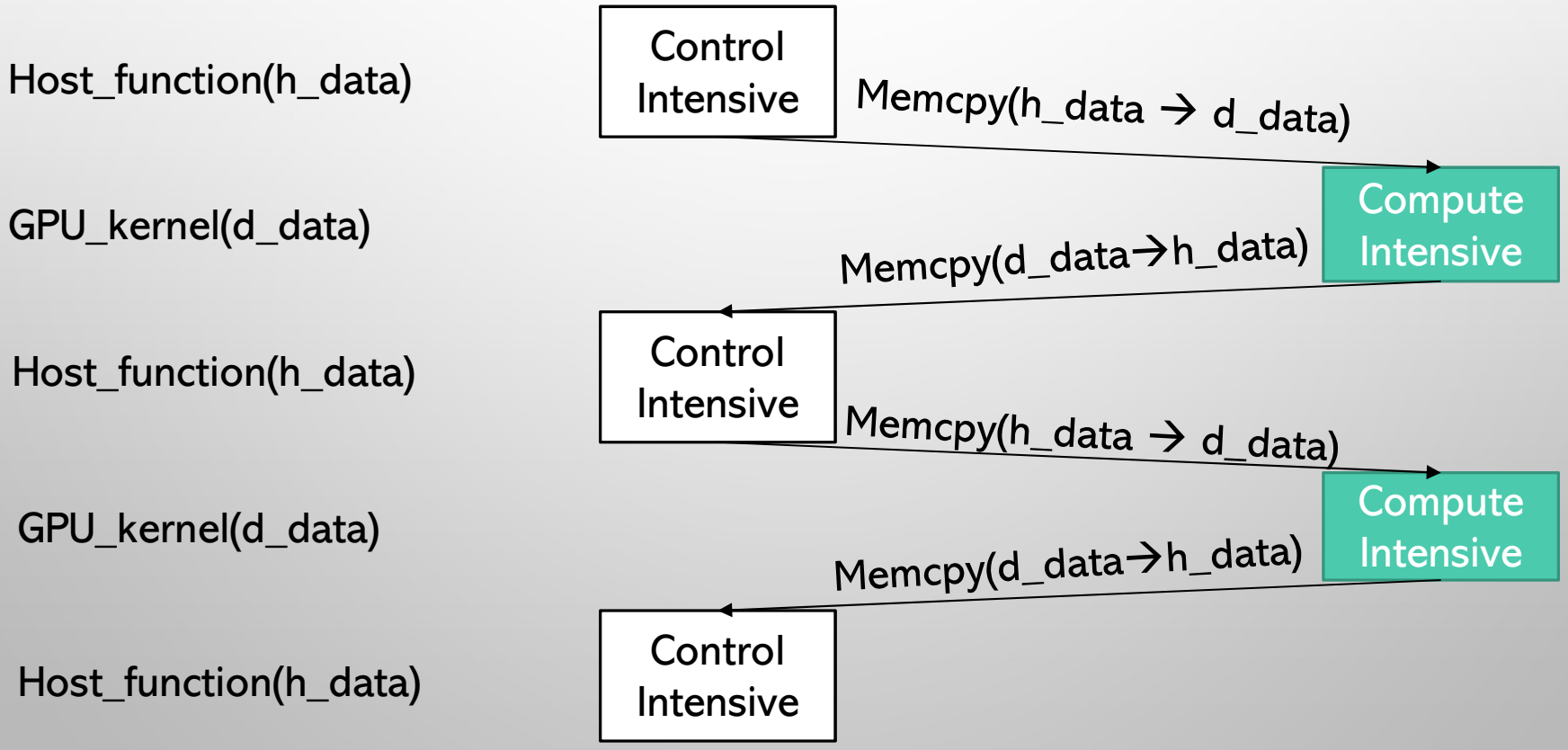
CPU-GPU WORKING MODEL



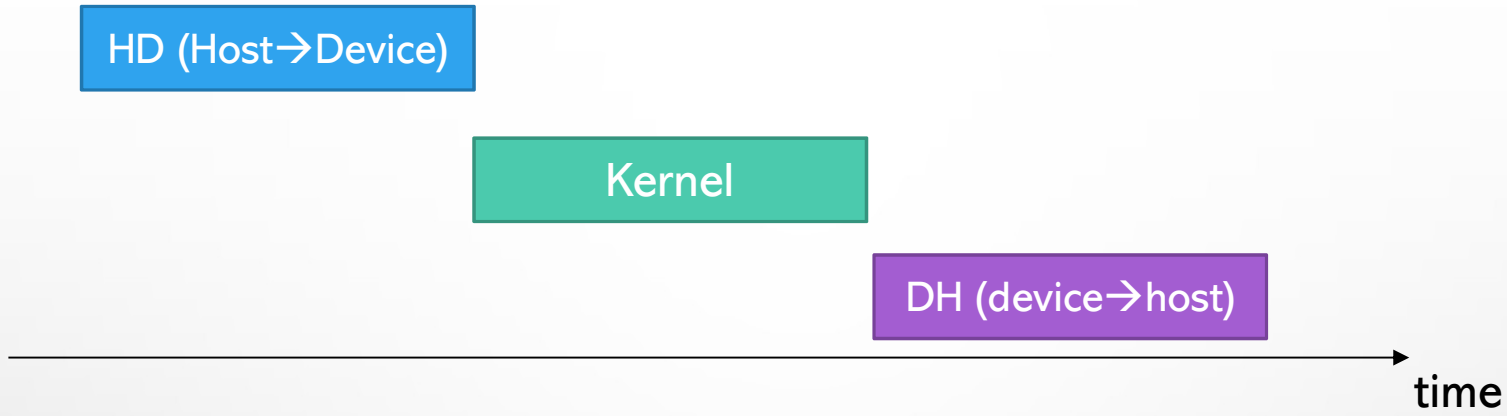
CPU-dGPU

'd' stands for dedicate, as opposed to integrated (iGPU)





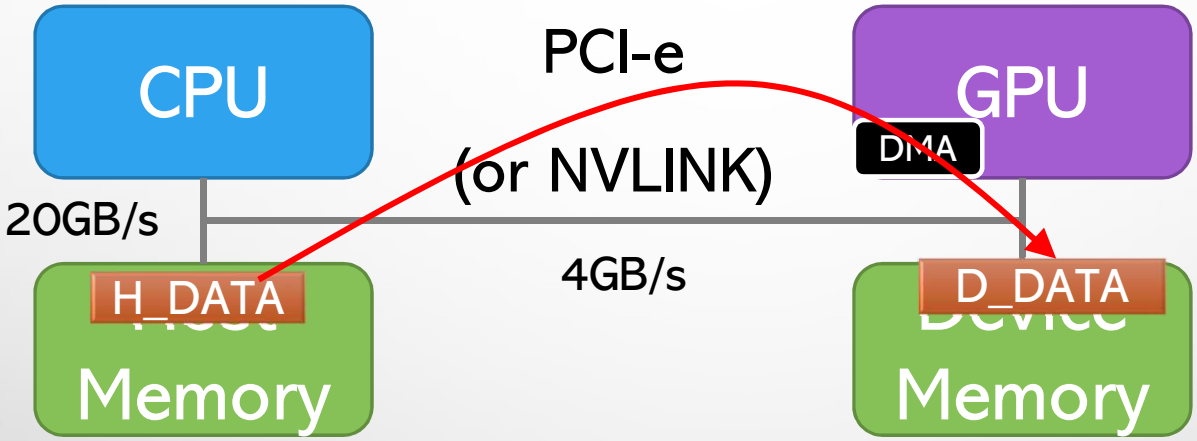
MEMCPY TIME



- For 8192x8192 transpose
  - HD: 44ms
  - Kernel: 2.3ms
  - DH: 44ms

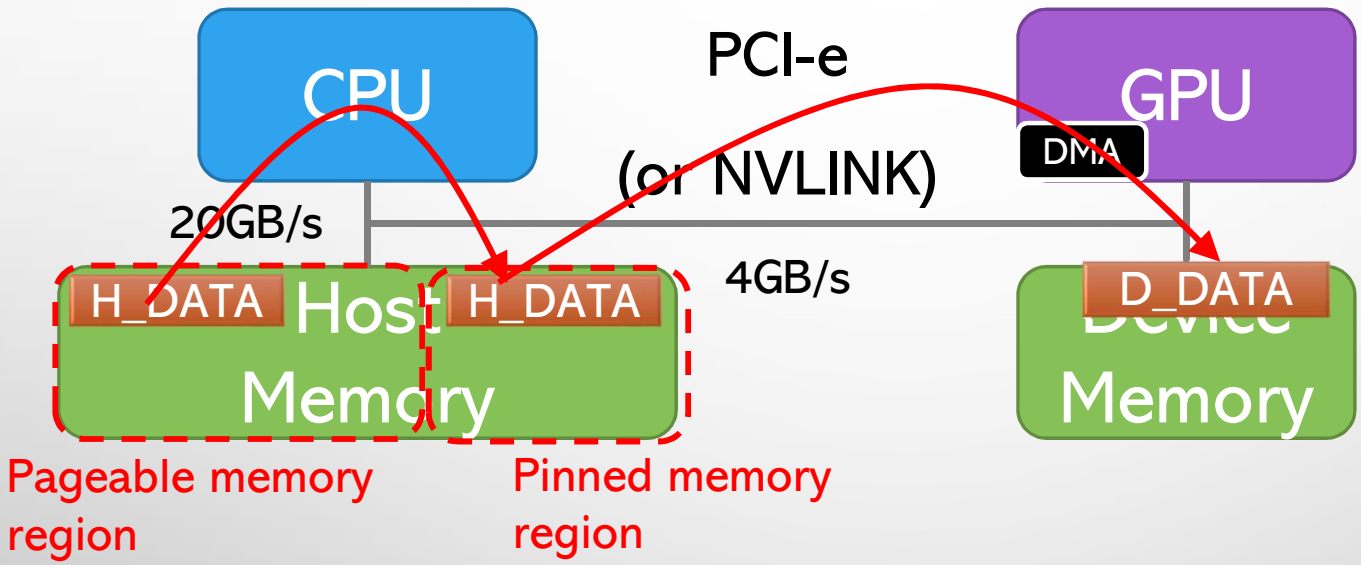
- For 2048x2048 matmul
  - HD (A,B) : 5ms
  - kernel : 25ms
  - DH (C) : 2.5ms
- Can we reduce this overhead?

HOW MEMCPY IS DONE



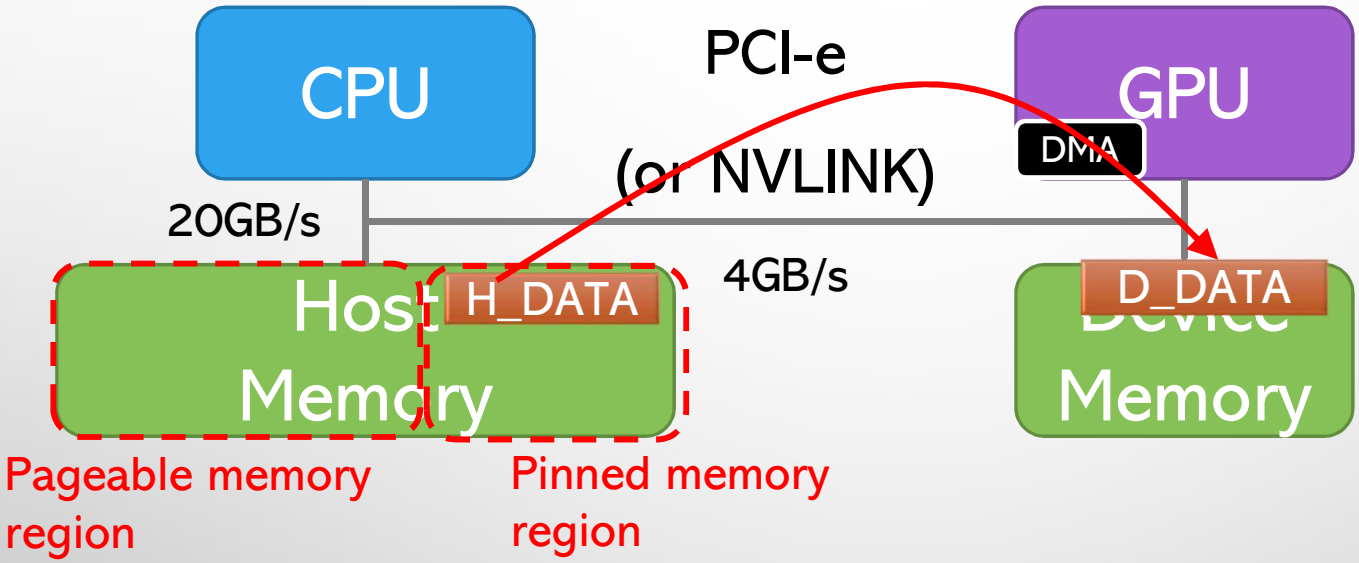
- DMA (Direct Memory Access)
- No need for CPU involvement

HOW MEMCPY IS DONE IN REALITY



- CPU needs to copy data into 'pinned' memory for DMA
- Why?
- The page might be residing on disk
  - DMA cannot access **page table**…
- OS keeps the 'pinned' region within physical memory

HOW MEMCPY IS DONE IN REALITY



- Then why don't we keep the data in the pinned memory in the first place?
  - It's a limited resource

USE PINNED MEMORY

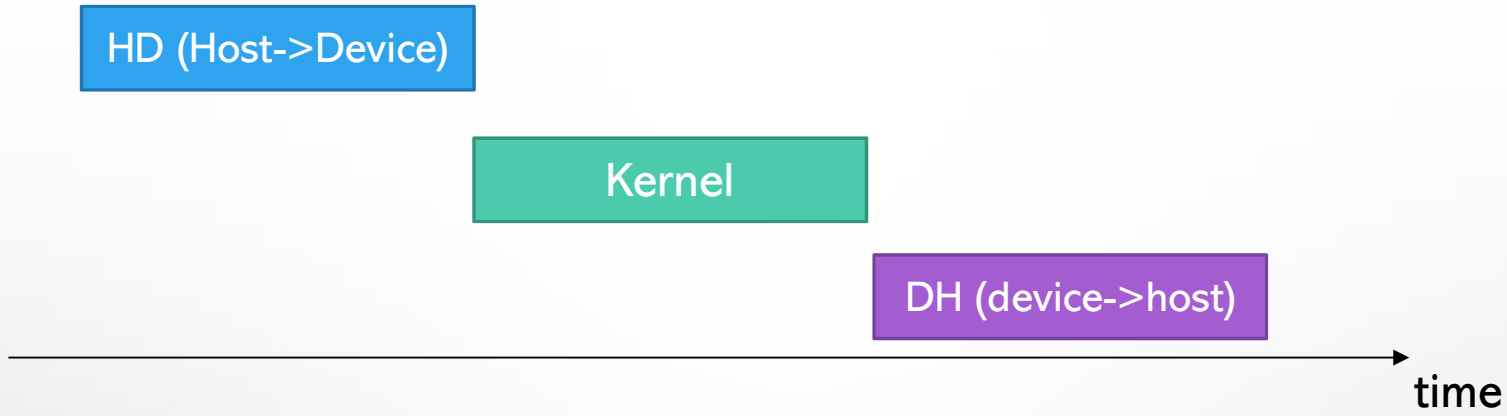
- cudaMallocHost (void** ptr, size_t size)

```
cudaMallocHost(&h_ptr,bytes);
```

- cudaHostRegister (void* ptr, size_t size, unsigned int flags)

```
h_ptr=malloc(bytes);
cudaHostRegister(h_ptr,bytes,0);
```

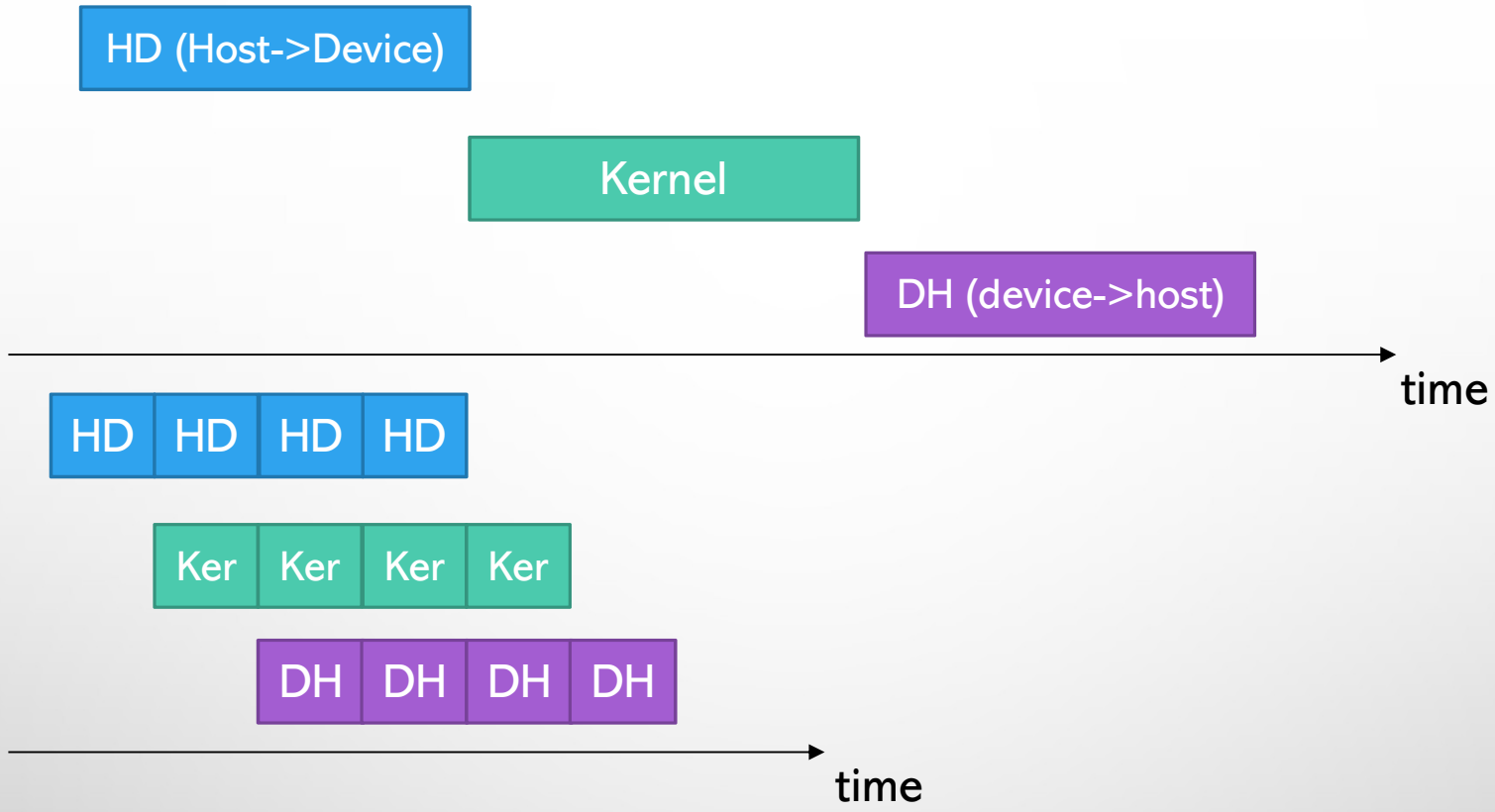
MEMCPY TIME WITH PINNED MEMORY



- For 8192x8192 transpose
  - HD: 44ms → **20ms**
  - Kernel: 2.3ms
  - DH: 44ms → **20ms**

- For 2048x2048 matmul
  - HD (A,B) : 5ms → **2.5ms**
  - kernel : 25ms
  - DH (C) : 2.5ms → **1.3ms**
- Copy bandwidth: 6.5GBps → 13GBps compared to unpinned memory
  - Similar speedup on Jetson (slightly lower speedup)

PIPELINING MEMCPY AND KERNELS



- Memcpy and kernel do not use same resource
- Meaning that we can pipeline them
- When you have to transpose/multiply 4 different matrices
- When you can split the kernel into multiples
  - Applies for both of transpose and matmul

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

 cudaStreamCreate(&stream1);
 cudaStreamCreate(&stream2);
 cudaStreamCreate(&stream3);
 cudaStreamCreate(&stream4);

 cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
 cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
 cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
 cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);
 //(I omitted copying the other matrix 'N' for space)

 MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

 cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
 cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
 cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
 cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);
cudaStreamCreate(&stream3);
cudaStreamCreate(&stream4);

cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);


MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

Use cudaStream to handle concurrency

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);
cudaStreamCreate(&stream3);
cudaStreamCreate(&stream4);

cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);


MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

- cudaMemcpyAsync allows concurrent copy

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);
cudaStreamCreate(&stream3);
cudaStreamCreate(&stream4);

cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);

MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

- cudaMemcpyAsync allows concurrent copy
- needs **PINNED** host memory

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);
cudaStreamCreate(&stream3);
cudaStreamCreate(&stream4);

cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);

MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

Put stream id next to the dynamic shared memory size

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

 cudaStreamCreate(&stream1);
 cudaStreamCreate(&stream2);
 cudaStreamCreate(&stream3);
 cudaStreamCreate(&stream4);

 cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
 cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
 cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
 cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);


 MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

 cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
 cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
 cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
 cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

PIPELINE USING STREAM

```
cudaStream_t stream1, stream2, stream3, stream4;

 cudaStreamCreate(&stream1);
 cudaStreamCreate(&stream2);
 cudaStreamCreate(&stream3);
 cudaStreamCreate(&stream4);

 cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
 cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
 cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
 cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);


 MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
 MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

 cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
 cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
 cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
 cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

Operations within a stream are serialized
Non-stream calls belong to 'default stream'

PIPELINE USING STREAM

```
cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);


MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);

cudaStreamSynchronize ( stream1);
```

If you want to make sure tasks within a stream are complete

PIPELINE USING STREAM

```
cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);


MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, …
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, …

cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);

cudaDeviceSynchronize ( );
```

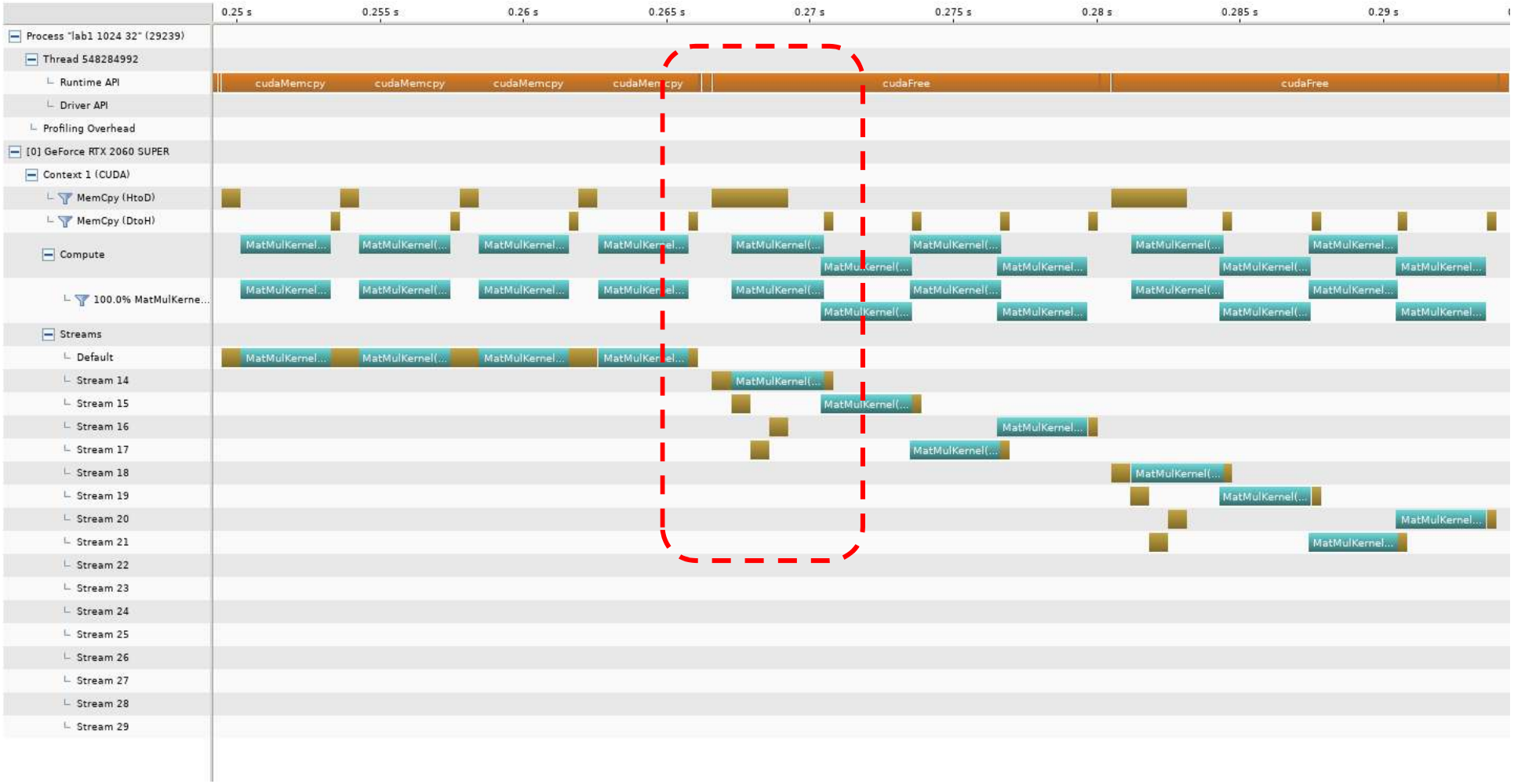
If you want to make sure tasks ~~within a stream~~ are complete

PIPELINE USING STREAM

```
cudaMemcpyAsync(d_M1.elements, M1.elements, matSize, cudaMemcpyHostToDevice, stream1);
cudaMemcpyAsync(d_N1.elements, N1.elements, matSize, cudaMemcpyHostToDevice, stream1);
MatMulKernel<<<dimGrid, dimBlock, 0, stream1>>>(d_M1.elements, d_N1.elements, d_P1.elements, M1.width);
cudaMemcpyAsync(P1.elements, d_P1.elements, matSize, cudaMemcpyDeviceToHost, stream1);

cudaMemcpyAsync(d_M2.elements, M2.elements, matSize, cudaMemcpyHostToDevice, stream2);
cudaMemcpyAsync(d_N2.elements, N2.elements, matSize, cudaMemcpyHostToDevice, stream2);
MatMulKernel<<<dimGrid, dimBlock, 0, stream2>>>(d_M2.elements, d_N2.elements, d_P2.elements, M2.width);
cudaMemcpyAsync(P2.elements, d_P2.elements, matSize, cudaMemcpyDeviceToHost, stream2);

cudaMemcpyAsync(d_M3.elements, M3.elements, matSize, cudaMemcpyHostToDevice, stream3);
cudaMemcpyAsync(d_N3.elements, N3.elements, matSize, cudaMemcpyHostToDevice, stream3);
MatMulKernel<<<dimGrid, dimBlock, 0, stream3>>>(d_M3.elements, d_N3.elements, d_P3.elements, M3.width);
cudaMemcpyAsync(P3.elements, d_P3.elements, matSize, cudaMemcpyDeviceToHost, stream3);

cudaMemcpyAsync(d_M4.elements, M4.elements, matSize, cudaMemcpyHostToDevice, stream4);
cudaMemcpyAsync(d_N4.elements, N4.elements, matSize, cudaMemcpyHostToDevice, stream4);
MatMulKernel<<<dimGrid, dimBlock, 0, stream4>>>(d_M4.elements, d_N4.elements, d_P4.elements, M4.width);
cudaMemcpyAsync(P4.elements, d_P4.elements, matSize, cudaMemcpyDeviceToHost, stream4);
```

The order does not matter (it did in the past)

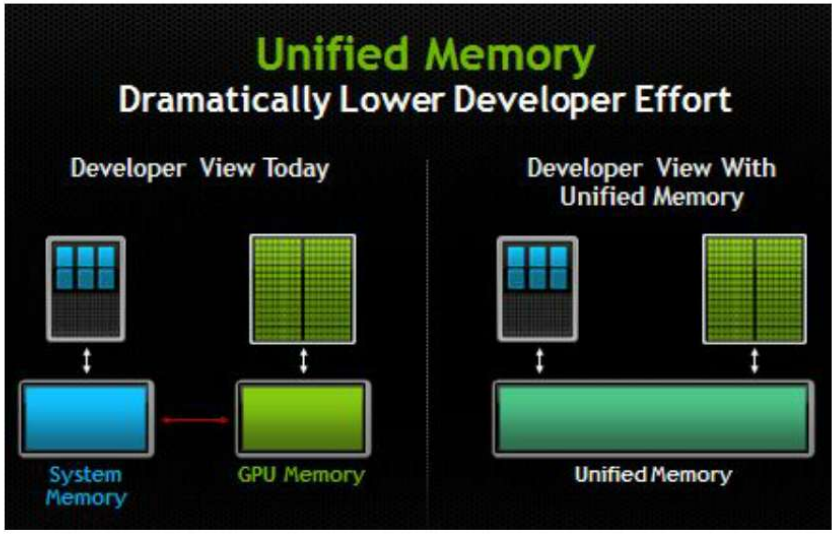
CHECK FROM A VISUAL PROFILER



CHECK FROM A VISUAL PROFILER

UNIFIED MEMORY

"Unified memory" or **"Managed memory"**:



UM Introduced in CUDA 6

Simpler Programming & Memory Model

Instead of two parallel pointers for same data
→ Single allocation, single pointer, somehow usable in host code or device code
→ Eliminate need for *explicit* copy

CUDA abstraction of host and device memory on programming level

→ i.e. **"developer view"** of memory model is as though there is only one shared memory between host and device. Still two distinct physical memories.

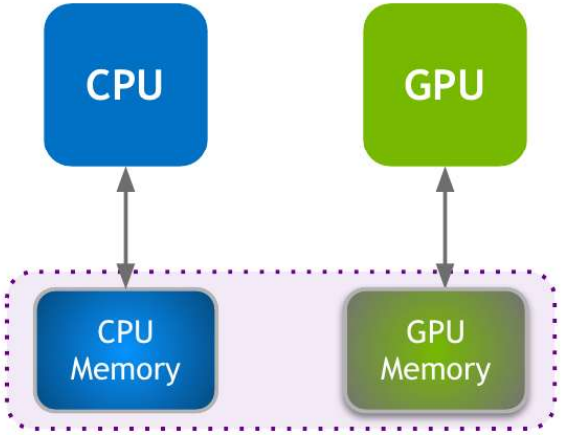
- Same *virtual* address space, and thus same pointer for both memories

Jetson has *physically* unified memory

Unified memory: one term that could refer to two different concepts

Discrete GPU

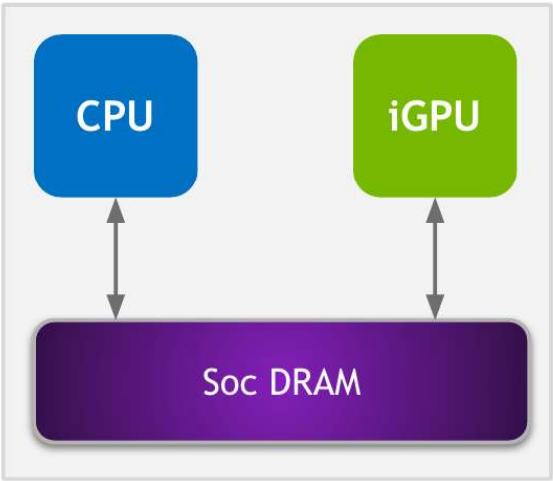
- CUDA Abstraction to simplify life of the programmer



"Unified memory"
from the developer's view

Jetson

- Single physical memory resource (DRAM) shared by CPU and GPU
- iGPU: Integrated GPU



Unified memory in
Jetson architecture

USE PINNED MEMORY

- cudaMallocHost (void** ptr, size_t size)

```
cudaMallocHost(&h_ptr,bytes);
```

- cudaHostRegister (void* ptr, size_t size, unsigned int flags)

```
h_ptr=malloc(bytes);
cudaHostRegister(h_ptr,bytes,0);
```

USE UNIFIED MEMORY

- cudaMallocManaged (void** ptr, size_t size)

```
cudaMallocManaged(&h_ptr,bytes);
```

- In Jetson, pinned memory and unified memory are almost the same
- <u>They can be accessed by both CPU and the GPU</u>

VECTOR ADDITION

```
float *A_h, *B_h, *C_h;//assume allocated and initialized
float *A_d, *B_d, *C_d;
cudaMalloc((void **) &A_d, size);
cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);
cudaMalloc((void **) &B_d, size);
cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);

cudaMalloc((void **) &C_d, size);

// Kernel invocation code – to be shown later
vecAddKernel<<<ceil(N/256.0), 256>>>(A_d, B_d, C_d, N);

// Transfer C from device to host
cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);
cudaFree(A_d); cudaFree(B_d); cudaFree (C_d);
```

VECTOR ADDITION (PINNED)

```
float *A, *B, *C;
~~float *A_d, *B_d, *C_d;~~
cudaMallocHost((void **) &A, size);
~~cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);~~
cudaMallocHost((void **) &B, size);
~~cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);~~
//init A and B… (code omitted)
cudaMallocHost((void **) &C, size);

// Kernel invocation code – to be shown later
vecAddKernel<<<ceil(N/256.0), 256>>>(A, B, C, N);

// No Transfer needed
~~cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);~~
cudaFreeHost(A); cudaFreeHost(B); cudaFreeHost (C);
```

VECTOR ADDITION (UNIFIED)

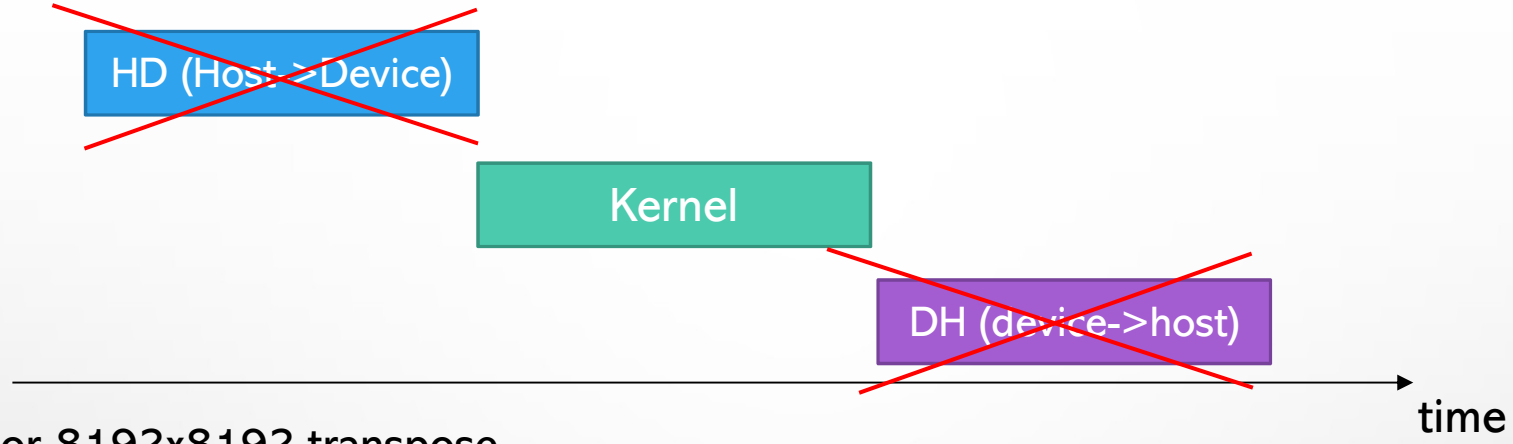
```
float *A, *B, *C;
~~float *A_d, *B_d, *C_d;~~
cudaMallocManaged ((void **) &A, size);
~~cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);~~
cudaMallocManaged ((void **) &B, size);
~~cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);~~
//init A and B… (code omitted)
cudaMallocManaged ((void **) &C, size);

// Kernel invocation code – to be shown later
vecAddKernel<<<ceil(N/256.0), 256>>>(A, B, C, N);
cudaDeviceSynchronize(); // make data visible to DPU
// No Transfer needed
~~cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);~~
cudaFree(A); cudaFree(B); cudaFree (C);
```

VECTOR ADDITION (UNIFIED)

```
float *A, *B, *C;
~~float *A_d, *B_d, *C_d;~~
cudaMallocManaged ((void **) &A, size);
~~cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);~~
cudaMallocManaged ((void **) &B, size);
~~cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);~~
//init A and B… (code omitted)
cudaMallocManaged ((void **) &C, size);

// Kernel invocation code – to be shown later
vecAddKernel<<<ceil(N/256.0), 256>>>(A, B, C, N);
cudaDeviceSynchronize(); // make data visible to DPU
// No Transfer needed
~~cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);~~
cudaFree(A); cudaFree(B); cudaFree (C);
```

This is needed in some devices
(Seems to be needed for Jetson)

EXECUTION TIME WITH NO COPY



- For 8192x8192 transpose
  - ~~HD: 44ms → 20ms~~
  - Kernel: 2.3ms
  - ~~DH: 44ms → 20ms~~
- For 2048x2048 matmul
  - ~~HD (A,B) : 5ms → 2.5ms~~
  - kernel : 25ms
  - ~~DH (C) : 2.5ms → 1.3ms~~

- Performance difference between pinned and unified memories is marginal in my experience
  - This might be different to you, or for other devices
  - The only way to find out is to measure

CACHE BEHAVIOR

- Pinned memory is uncached, unified memory is cached

Memory Type	CPU	iGPU
Device memory	Not directly accessible	Cached
Pageable host memory	Cached	Not directly accessible
Pinned host memory	Uncached where compute capability is less than 7.2. Cached where compute capability is greater than or equal to 7.2.	Uncached
Unified memory	Cached	Cached

- The code is identical for unified memory between iGPU and dGPU
- The code is different for pinned memory between iGPU (no memcpy) and dGPU (memcpy needed)
- Cache coherency
  - One-way coherency (GPU sees CPU changes, not the other direction)
  - GPU → CPU is handled by cache flush
  - → If you mix CPU and GPU codes, it will be very slow…
  - Not the magic we were expecting (yet)

SUMMARY

- In Jetson, you don't need to memcpy unless you really need another copy for GPU to work on
- cudaMallocHost() for pinned memory
- cudaMallocManaged() for unified memory

- Use cudaStream for concurrency
  - Overlapping memcpy and computation
  - Concurrent execution of multiple kernels